Installazione WSL per chi ha windows (1h)

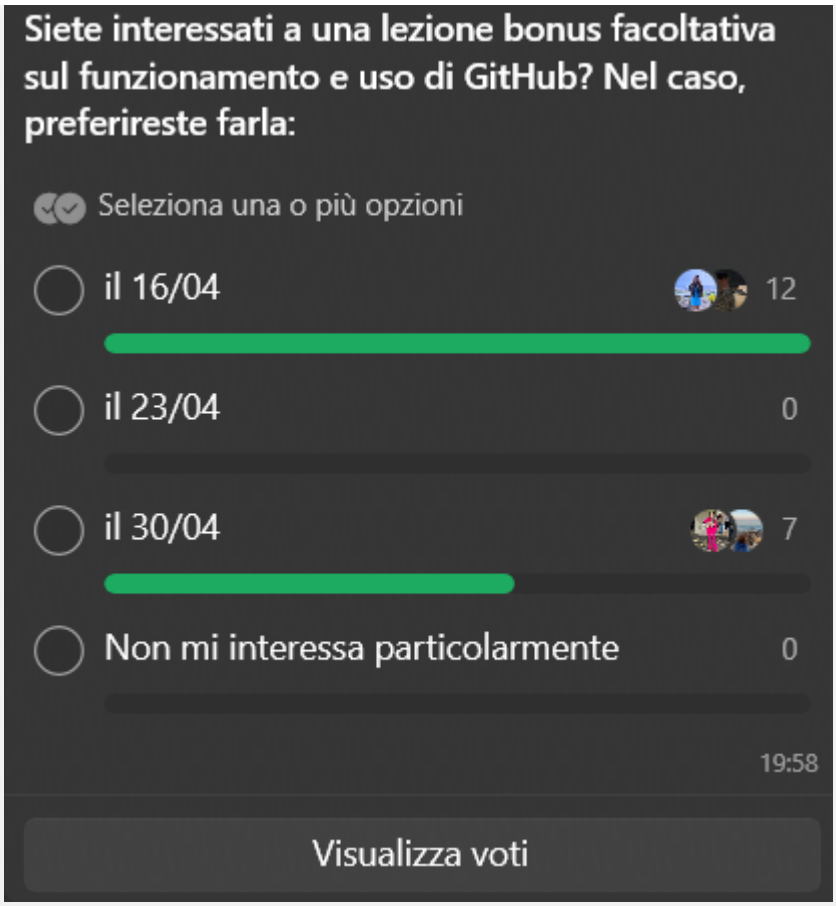
[[Programming Tools and Python Installation Guide]]

INZIO

Spiegazione Struttura Corso

Struttura del corso:

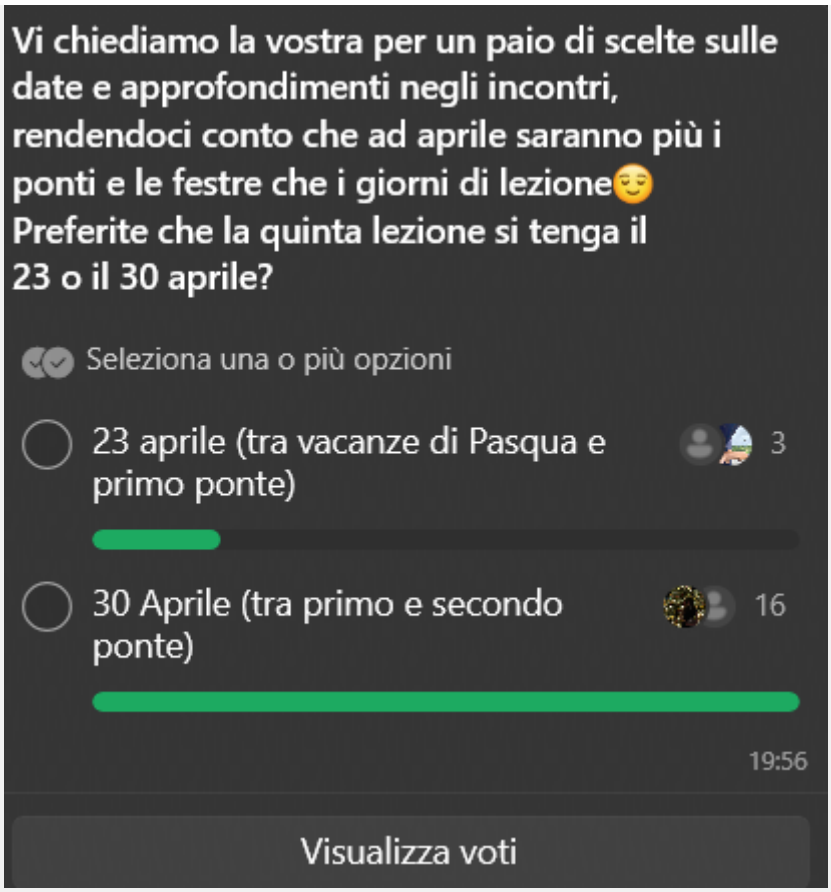
> [!NOTE]- Lezioni, date e argomenti
>
> **Lezione 1 +1h Extra (19/03)** [[Lezione 1 - Python Course]]
>
> - Installazione WSL per chi ha windows (1h)
> - **INZIO**
> - Spiegazione Struttura Corso+ Progetto finale pytorch
> - Perchè Linux
> - Installazione e Aggiornamento Terminali (Ubuntu e MAC)
> - Comandi Terminali
> - Visual Studio
> - Introduzione a Python
> - Python da Terminale
> - Guest?
> - Culture Pill/esterni
>
> **Lezione 2 (26/03)**
>
> - Data Types 1 e relativi metodi (Test su Python da Terminale):
> - If/else/elif
> - Data Types 2 e relativi metodi:
> - For Loops
> - Culture Pill/esterni
>
> **Lezione 3 (2/04)**
>
> - Data Types 3
> - Funzioni 1
> - While loops
> - Data Types 4:
> - Culture Pill/esterni
>
> **GitHub Forse (16/04)**



- ... **Lezione 4 (23/04)**
- Funzioni 2
- Classi 1
- Ambienti Virtuali
- numpy

Lezione 5 (30/04)

- Classi 2
- Error Handling
- File esterni
- Creare una Libreria



- Culture Pill/esterni

DECIDERE PROGETTO COSA FARE E MODIFICARE PROSSIME LEZIONI DI CONSEGUENZA

Lezione 6 (7/05) (data Analysis 101)

- Pandas
- Matplotlib
- Numpy 2
- Culture Pill/esterni

Lezione 7 (14/05)

- Jupyter Notebook
- PyTorch
- Culture Pill/esterni

Lezione 8 (21/05)

- Progetto
- Culture Pill/esterni

Lezione 9 (28/05)

- Progetto
- Culture Pill/esterni
- Saluti

Progetto finale in Pytorch

Tempistiche (potremmo sforare)

Perchè Linux

- Perchè per progetti embedded si userà quello
- perchè è il più versatile
- perchè è uno standard
- perchè ha una curva di apprendimento più difficile ed è meglio affrontarlo con qualcuno, da soli ci si trova disorientati.
- Nel campo biomedico, molti dispositivi e strumenti diagnostici implementano sistemi operativi Linux, sfruttandone la stabilità e la versatilità.
- L'ecosistema open-source di Linux offre grande trasparenza e possibilità di personalizzazione, una caratteristica importante per la ricerca.

Esempi di linux in ambito biomedico

1. Sistemi di analisi e modellazione di dati clinici su Linux in ambiente HPC (High Performance Computing)
    - In un laboratorio di ricerca biomedica, capita di dover processare grandi moli di dati, come immagini di risonanza magnetica (MRI), dati di genomica o sequenziamenti di DNA (NGS). Per elaborare questi dataset in modo efficiente, si utilizzano server Linux dotati di GPU e altre risorse ad alte prestazioni.
    - I software bioinformatici più usati (ad esempio, diversi tool per l'analisi del DNA e dell'RNA) sono spesso sviluppati o ottimizzati per Linux, il che riduce problemi di compatibilità e aumenta la stabilità dell'ambiente di lavoro.
    - La facilità di automazione (tramite script Bash o Python) rende Linux molto adatto a pipeline complesse di analisi, in cui i dati passano attraverso vari passaggi (pulizia, filtraggio, allineamento, analisi statistica, ecc.).
2. Utilizzo di Raspberry Pi o sistemi embedded basati su Linux per dispositivi di monitoraggio o analisi diagnostica
    - Un Raspberry Pi o un'altra scheda embedded (come BeagleBone, Jetson Nano, o simili) può essere impiegato come cervello "low cost" di piccoli dispositivi biomedici, ad esempio per monitorare parametri vitali di un paziente.
    - Nel campo della telemedicina, un sistema embedded Linux può raccogliere dati tramite sensori (pressione arteriosa, battito cardiaco, temperatura corporea) e inviarli in modo sicuro a un server remoto, permettendo un monitoraggio costante anche fuori dall'ospedale.
    - Grazie alla natura open-source di Linux, è possibile customizzare il sistema operativo e installare soltanto ciò che serve, ottimizzando prestazioni e consumo energetico. Questo risulta prezioso se il dispositivo deve funzionare a batteria o in contesti con risorse limitate.
3. Laboratori di ricerca che gestiscono grandi volumi di dati (sequenziamento del DNA, elaborazione di immagini mediche) con software ottimizzati per Linux
    - I laboratori che si occupano di genomica, proteomica o imaging biologico utilizzano software specializzati (come BLAST, Bowtie, GATK per genetica, oppure ANTs, FSL per neuroimaging).

Questi tool sono spesso nativi per Linux o vantano performance migliori su Linux grazie a dipendenze e librerie ottimizzate.
- In molti casi, i laboratori dispongono di cluster di calcolo interni o si appoggiano a servizi cloud – quasi sempre basati su distribuzioni Linux – per eseguire analisi a elevato carico computazionale.
- Lavorare in Linux semplifica anche l'automazione di processi ripetitivi (con cron job, script Bash/Python, container Docker, ecc.), un vantaggio notevole in pipeline sperimentali lunghe e cicliche.

In sintesi, i punti chiave sono l'elevata efficienza, la flessibilità e la stabilità che Linux offre in molti settori biomedici: dalla ricerca di base (analisi bioinformatiche, HPC) allo sviluppo di piccoli dispositivi embedded e strumenti di telemedicina.

Cos'è ubuntu

Ubuntu è una distribuzione Linux particolarmente user-friendly e diffusa, ideale per chi desidera avvicinarsi al mondo Linux. Offre un ciclo di aggiornamenti stabile, un vasto supporto della community e un'ampia disponibilità di software precompilato, inclusi molti strumenti bioinformatici.

Installazione e Aggiornamento Terminali (Ubuntu e MAC)

Gestione dei pacchetti

Come gestire i pacchetti software da command line.

> [! warning] Attenzione! Per chi usa mac è necessario aver installato homebrew!

apt-get Command	Description	Homebrew Equivalent
`apt-get update`	Fetch and update the package lists from repositories	`brew update`
`apt-get install <package-name>`	Install a specific package	`brew install <package-name>`
`apt-get remove <package-name>`	Remove a specific package without removing its configuration files	`brew uninstall <package-name>`
`apt-get purge <package-name>`	Remove a specific package and its configuration files	`brew uninstall <package-name>`
`apt-get upgrade`	Upgrade all packages to the newest available version	`brew upgrade`
`apt-get autoremove`	Remove packages that were automatically installed to satisfy dependencies and are no longer needed	`brew cleanup`
`apt-get clean`	Clear downloaded archive files	`brew cleanup`
`dpkg -l`	List all installed packages	`brew list`
`apt-cache search <term>`	Search for a package by its name or description	`brew search <term>`

apt-get Command	Description	Homebrew Equivalent

DA FARE: AGGIORNARE I TERMINALI

UNIX 101

> [! important] Importante Questa **NON** è da seguire, passo passo. Serve per dare un minimo di background quando usiamo certi comandi. Per questo motivo **NON ESEGUIRE NESSUN COMANDO**, fino alla sezione successiva.

Percorsi

Nei nomi della cartelle e dei file è caldamente consigliato usare solo lettere, numeri e trattini alti e bassi. Altri caratteri sono spesso supportati ma richiederebbe avvolgere tra virgolette i nomi ogni volta e ci impazzite dietro.

I seguenti indirizzi possono essere usati come argomento di altri comandi come `cd`, `touch`, `mkdir` e molti altri che abbiamo visto o vedremo.

Immaginiamo la seguente struttura:

undefined Vediamo quali sono i percorsi per arrivare alle diverse cartelle, supponendo di trovarci dentro la cartella *this_folder*.

This folder:

```
.
```

Next folder:

```
./next_folder_name
```

Parent folder:

```
..
```

Parent of parent folder:

```
../..
```

Sibling folder:

```
../sibling_folder
```

File di testo nella cartella *sibling*:

```
../sibling_folder/text_file.txt
```

è importante sottolineare come l'estensione sia NECESSARIA quando si fa fa riferimento ad un file.

Cartella home:

```
~
```

che è un modo più rapido per scrivere:

```
/home/user_name
```

Root directory (è la radice del file-system):

```
/
```

Aprire la cartella `path_to_folder/folder_name` nell'app predefinita:

- Windows WSL:

```
wslview path_to_folder/folder_name
```

o alternativamente:

```
explorer.exe path_to_folder/folder_name
```

- Mac OS:

```
open path_to_folder/folder_name
```

Aprire visual studio code nella cartella corrente:

```
code .
```

Nota: per mac bisogna averlo aggiunto al path per farlo funzionare.

Navigare in una cartella

```
cd path_to_folder
```

Creare una cartella

```
mkdir path_to_folder/name_of_folder
```

Sapere il percorso completo alla cartella corrente:

```
pwd
```

Creare un file

```
touch path_to_file/file_name.extension
```

Come Installare e Aggiornare Pacchetti

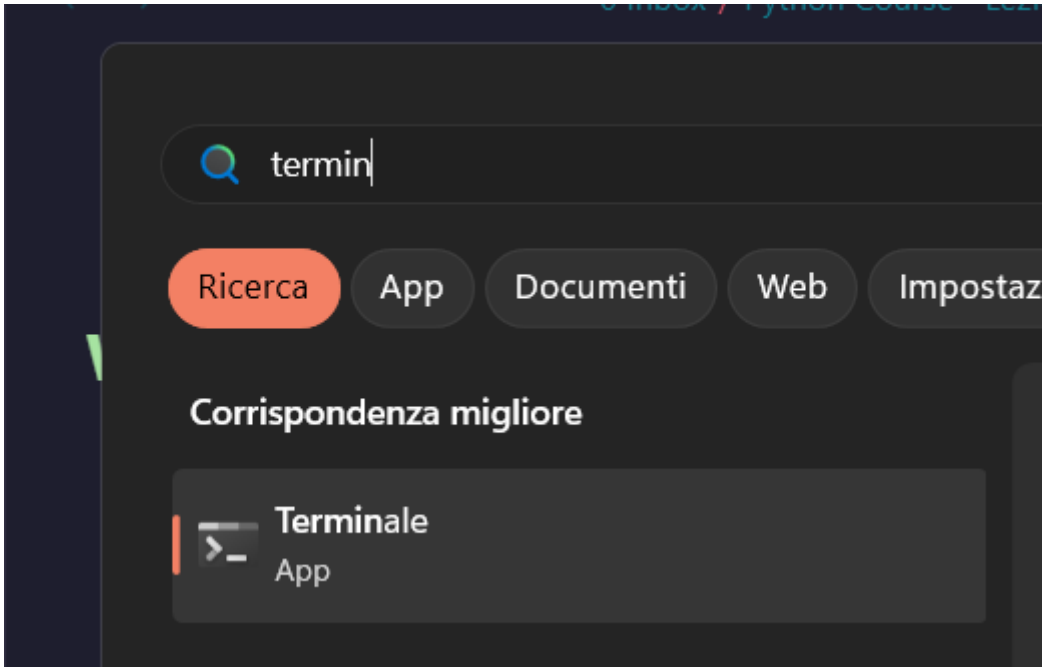
- Comandi Terminali
  - Comandi Semplici
  - Nano Editor
  - Editare un file, estensioni, aprire una cartella
- *Problema: Creare un file, navigare in una directory, eseguire un file*

Visual Studio

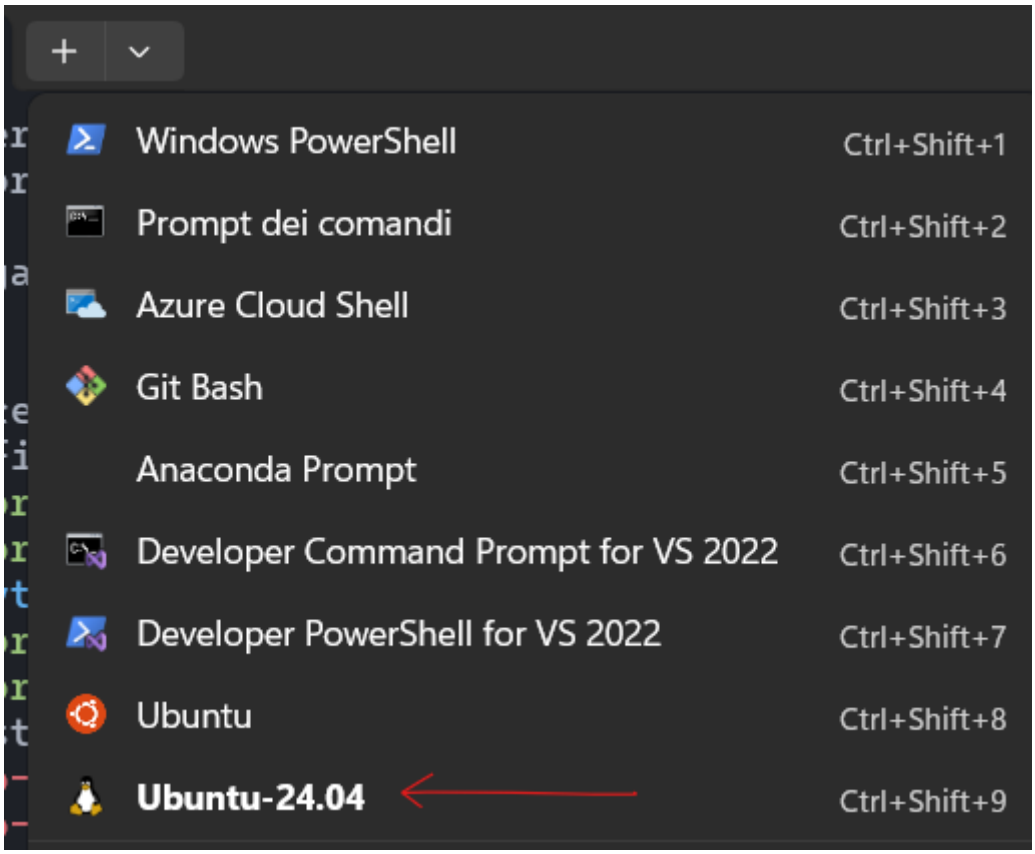
Windows Installation

1. Installa Visual Studio Code.

2. Cerca il terminale (`Windows + S`)



3. Scegli ubuntu 2024-4.1 LTS dal menù a tendina



4. Vai nella cartella python course: `cd python_course`
5. Avvia visual studio code in quella cartella `code .` Dovrebbe partire un download di un server che connette visual studio code a wsl, in questo modo visual studio code gira su windows modificando i dati in Linux

MAC Installation

1. Scarica Visual Studio Code per macOS.
2. Apri l'elenco dei download del browser e individua l'app o l'archivio scaricato.

3. Se è un archivio, estrai il contenuto. Usa il doppio clic per alcuni browser o seleziona l'icona della 'lente d'ingrandimento' con Safari.
4. Trascina `Visual Studio Code.app` nella cartella **Applicazioni**, rendendolo disponibile nel Launchpad di macOS.
5. Apri VS Code dalla cartella **Applicazioni**, facendo doppio clic sull'icona.

Puoi anche eseguire VS Code dal terminale digitando 'code' dopo averlo aggiunto al percorso:

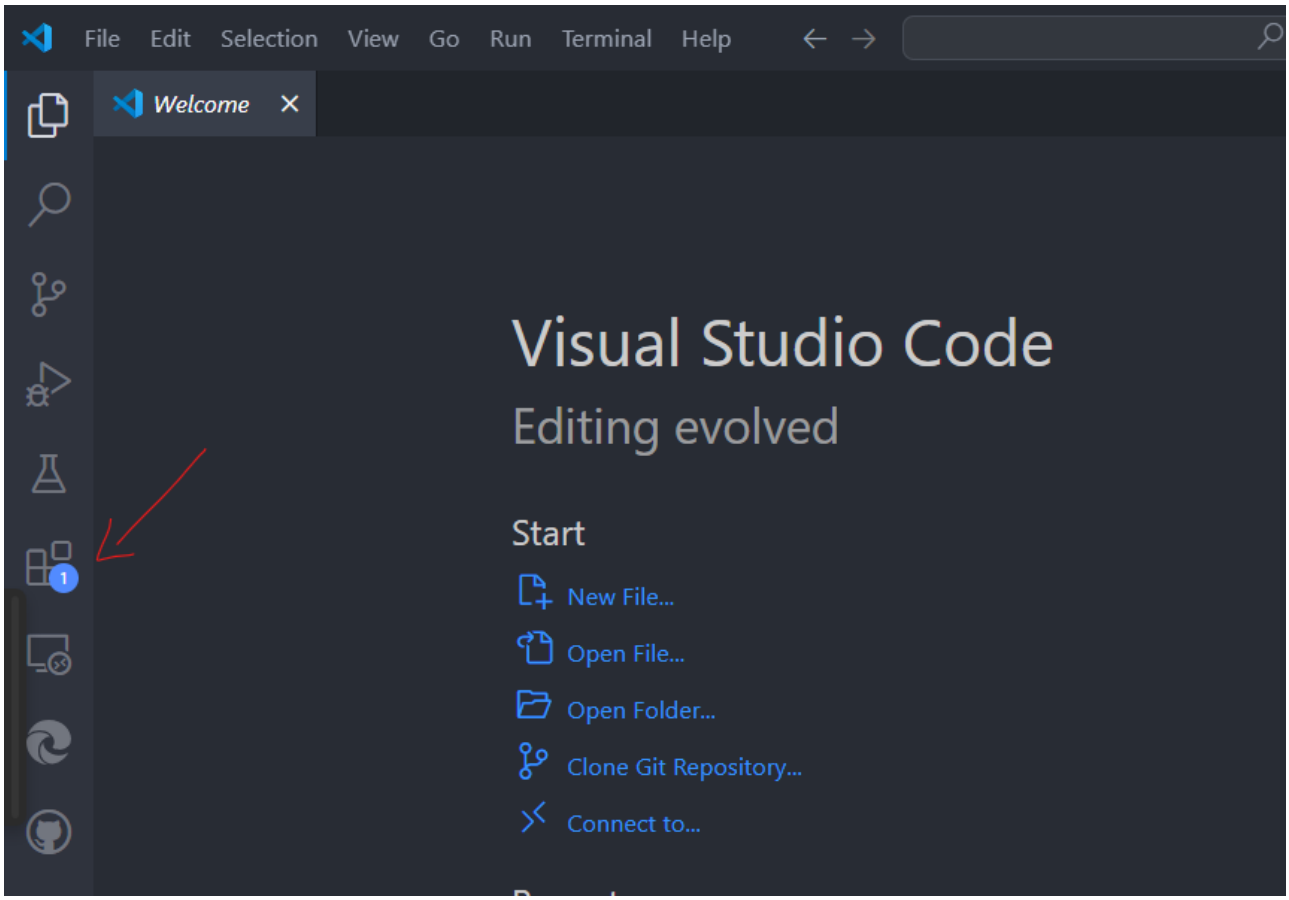
- Avvia VS Code.
- Apri la **Command Palette** (Cmd+Shift+P) e digita 'shell command' per trovare il comando **Shell Command: Install 'code' command in PATH**. undefined
- Riavvia il terminale affinché il nuovo valore `$PATH` abbia effetto. Potrai digitare 'code .' in qualsiasi cartella per iniziare a modificare i file in quella cartella.

> **Nota:** Se hai ancora il vecchio alias `code` nel tuo `.bash_profile` (o equivalente) da una versione precedente di VS Code, rimuovilo e sostituiscilo eseguendo il comando **Shell Command: Install 'code' command in PATH**.

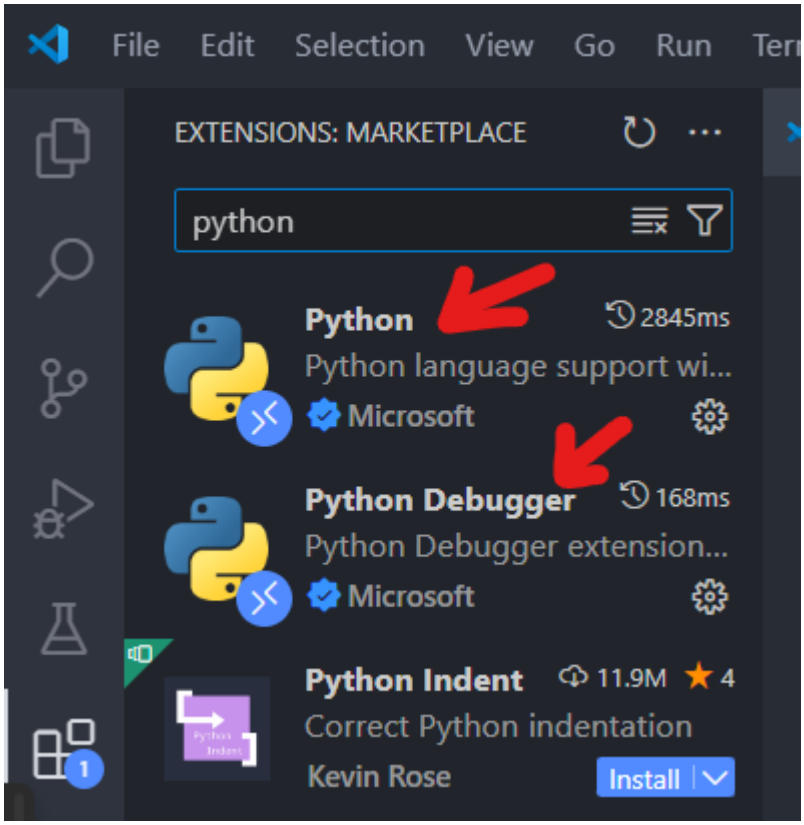
Se non funziona, provare a copiare ed incollare nel terminale questi comandi.

Install extensions

1. Aprire visual studio code dal terminale nella cartella che interessa (chi ha windows deve aprirlo dal terminale di wsl, vedi sopra come) `code .`
2. Cliccare nella barra a sinistra sull'icona estensioni



3. Cercare python nella barra di ricerca ed installare le seguenti estensioni



Aggiiungere: installare WSL extension chiudere e riaprire il terminale. Non installare da apt-get!! Dobbiamo usare la versione di windows!!

Introduzione a Python

1. *Cos'è Python*
   Python è un linguaggio di programmazione interpretato, multi-paradigma e ad alto livello. È apprezzato per la sua sintassi pulita e leggibile, che lo rende un'ottima scelta sia per principianti sia per sviluppatori esperti. In Python puoi scrivere script veloci per prototipare un'idea o sviluppare progetti di grandi dimensioni grazie alla sua vasta libreria standard e al supporto di numerosi framework.

2. *Perché Python*
   • **Semplicità e velocità di sviluppo**: La sintassi chiara e la ricca collezione di librerie permettono di realizzare progetti in meno tempo rispetto ad altri linguaggi.
   • **Comunità e risorse**: Python ha un'enorme community di sviluppatori, quindi è facile trovare guide e risolvere problemi, soprattutto nei campi più specialistici come la scienza dei dati e la bioinformatica.
   • **Versatilità**: Python si usa in tantissimi ambiti, dal web development (Django, Flask), all'analisi dati e Machine Learning (pandas, NumPy, scikit-learn, PyTorch), fino ad applicazioni embedded (seppure meno frequenti).
   • **Integrazione con altri linguaggi**: Python può interfacciarsi con librerie in C/C++ e Java, sfruttando così performance e compatibilità già esistenti.

3. *Quale Python*
   Attualmente, la versione più utilizzata e supportata è la Python 3.x (ad esempio, Python 3.10 o 3.11). Python 2.x è ormai deprecato e non riceve più aggiornamenti, quindi è sempre consigliabile usare Python 3 in qualsiasi nuovo progetto.
   • Python "vanilla": è la versione standard downloadabile dal sito ufficiale (python.org).

• Distribuzioni scientifiche: come Anaconda, che includono moltissimi pacchetti per la scienza dei dati e la ricerca (NumPy, SciPy, Matplotlib, ecc.). Questo risulta particolarmente pratico in ambito biomedico, perché hai già tutto a portata di mano per l'analisi di dati clinici.

4. *Come Python*
   • **Installazione:** In ambiente Linux/Ubuntu, è spesso pre-installato o facile da aggiungere tramite apt-get. In macOS si può installare con Homebrew o scaricando il pacchetto ufficiale, mentre in Windows è necessario scaricarlo dal sito ufficiale o tramite Microsoft Store (o usare WSL, come previsto dal corso).
   • **Utilizzo:** Puoi scrivere i tuoi script in un qualsiasi editor di testo, avviarli da riga di comando (python script.py) oppure usare un IDE come Visual Studio Code, PyCharm o anche Jupyter Notebook per analisi interattive.
   • **Virtual Environment:** In Python è raccomandabile gestire le dipendenze con virtual environment (venv) o ambienti conda, soprattutto se hai progetti diversi che richiedono versioni di librerie non compatibili fra loro.

5. *Esempi di Python in ambito biomedico*
   • **Analisi di dati clinici**: Python è molto utilizzato per analizzare dataset di pazienti, estrarre statistiche grazie a librerie di data science e anche per la creazione di modelli di Machine Learning in ambito diagnostico o predittivo (ad esempio, previsione di complicanze post-operatorie).
   • **Elaborazione di immagini medicali**: Con OpenCV, SimpleITK o librerie di deep learning, si possono processare immagini di TAC, risonanze magnetiche o radiografie. Python facilita la prototipazione rapida di algoritmi per la segmentazione o il riconoscimento di patologie.
   • **Modellazione e simulazioni:** Per la modellazione computazionale di processi biologici (dalla struttura proteica fino alla cinetica di farmaci), esistono librerie in Python che semplificano la creazione e il test di modelli matematici e statistici.
   • **Bioinformatica:** Librerie come Biopython aiutano nell'analisi di sequenze di DNA/RNA, nell'interazione con database di proteine e di geni e nel parsing di formati di file standard come FASTA, GenBank, ecc.

6. *Pro e contro di Python*

- Punti di forza:
  - Facile da imparare e molto leggibile, ideale per prototipazione rapida.
  - Eccellente ecosistema di librerie (soprattutto per data science e bioinformatica).
  - Community e documentazione vastissime, risolve i problemi più comuni in modo immediato.

- Limiti e svantaggi:
  - **Performance**: Python è solitamente più lento di C++ o Java. In molti casi, tuttavia, questo deficit si supera accedendo a librerie ottimizzate in C/C++ (la parte più pesante gira in nativo, mentre Python funge "solo" da interfaccia).
  - **Non è il più adatto agli ambienti embedded con memoria molto ridotta**, anche se esiste MicroPython per dispositivi particolarmente limitati.
  - La gestione dell'ambiente (versioni e dipendenze) può diventare complessa se non ci si organizza bene, soprattutto quando si lavora su più progetti contemporaneamente.

In sintesi, Python si rivela un linguaggio flessibile, potente e molto comprensibile, con una curva di apprendimento amichevole e un ecosistema di strumenti concepiti per la ricerca scientifica e il settore biomedico.

Python da Terminale

Per utilizzare Python da terminale, innanzitutto devi avere Python installato e disponibile nel tuo percorso di sistema (in Windows, macOS o Linux). Una volta soddisfatto questo requisito, puoi aprire il tuo terminale (o la shell di WSL, se stai usando Windows Subsystem for Linux) e lanciare direttamente comandi Python. Ecco alcune operazioni comuni:

• Avvio dell'interprete interattivo:
– Basta digitare "python" (o "python3", a seconda della configurazione) per entrare in una sessione interattiva. Puoi testare rapidamente uno snippet di codice, fare calcoli veloci o sperimentare con piccole funzioni.

> [!a cosa serve l'interprete]- L'interprete interattivo di Python (cioè lanciare il comando "python" o "python3" nel terminale) può risultare molto utile in diversi scenari, spesso in maniera più comoda rispetto allo scrivere un intero programma in un file. Ecco alcuni motivi:
>
> 1. Sperimentazione Rapida:
> • Se vuoi testare velocemente un'idea o una singola funzione, puoi digitare il codice riga per riga e vedere immediatamente come si comporta. È l'ideale per calcoli veloci, piccoli frammenti di logica o per fare debug su qualche funzione.
> • È un po' come usare una "calcolatrice potenziata" o blocchi di codice di prova senza dover creare ogni volta un file nuovo.
>
> 2. Debug e Prototipazione:
> • Quando scrivi un programma più vasto, se ti serve controllare un certo valore o provare una parte di codice, puoi entrare nell'interprete e richiamare le stesse funzioni o classi.
> • Invece di modificare continuamente il file principale, salvalo, eseguilo e stampane l'output, puoi provare i cambiamenti "sul momento" nell'interprete, accelerando il ciclo di sviluppo.
>
> 3. Automazione Versatile:
> • L'interprete è comodo da integrare in script Bash o nei cron job (processi programmati su Linux/WSL). Puoi eseguire comandi Python come fossero comandi di sistema e creare pipeline di elaborazione dati.
> • Se devi eseguire un singolo script veloce (ad esempio rinominare vari file in una cartella o fare un piccolo parsing di testo), non è necessario un intero "programma con interfaccia": basta uno script di poche righe lanciato dal terminale.
>
> 4. Apprendimento e Test Didattici:
> • Per chi è alle prime armi, l'interprete aiuta a prendere confidenza con la sintassi e i concetti base di Python senza "sovrastrutture". Scrivi una riga e vedi subito l'esito.
> • Molti tutorial e documentazioni mostrano esempi riga per riga proprio in questo formato, facilitando l'interazione diretta.
>
> 5. Librerie che Richiedono Interattività:
> • Alcune librerie (come quelle di data science, NumPy, pandas, Matplotlib) si prestano a essere

> provate al volo nell'interprete, soprattutto se vuoi soltanto esplorare un dataset o generare un grafico prototipo.
>
> In conclusione, l'interprete Python da terminale è un ottimo strumento quando:
> • Devi "fare due prove" veloci.
> • Vuoi sfruttare Python come se fosse un ambiente di calcolo interattivo (tipo "calcolatrice intelligente").
> • Hai bisogno di un debug immediato o di un piccolo scripting per lavorare con file, cartelle e dati senza gestire un intero progetto.
>
> Se invece il progetto è di più ampio respiro, vorrai certamente scrivere uno o più file .py ben strutturati. Ma l'interprete rimane comunque un alleato indispensabile per test rapidi e per la fase di studio o debugging.

• Esecuzione di script Python:
– Se hai un file "script.py", puoi eseguirlo con "python script.py". Questa modalità è ideale per script più lunghi che richiedono vari argomenti, elaborazioni e output multipli.

• Passaggio di argomenti da riga di comando:
– Se il tuo script accetta argomenti (ad esempio, "python script.py --option value"), puoi recuperarli nel codice Python con il modulo "sys.argv" o con librerie come "argparse" per gestioni più complesse.

• Installazione di moduli e librerie aggiuntive:
– Da terminale, puoi installare pacchetti Python con "pip install nome_pacchetto" (oppure "conda install nome_pacchetto" se stai usando Anaconda). In questo modo, puoi arricchire il tuo ambiente con librerie specializzate, come quelle per l'analisi di dati clinici o l'elaborazione di immagini biomediche.

• Perché usare il terminale:
– Automatizzare flussi di lavoro: puoi lanciare i tuoi script Python in sequenza o in combinazione con altri comandi UNIX per creare pipeline di elaborazione dati.
– Integrazione con cron job o script Bash: se vuoi programmare un'operazione periodica (ad esempio un'analisi notturna di nuovi dataset), usare Python da riga di comando ti permette di integrarlo senza problemi in script più ampi.
– Debug rapido: se devi testare o provare piccole modifiche al volo puoi entrare nell'interprete interattivo o eseguire uno script senza aprire un IDE completo.

In sostanza, usare Python da terminale risulta molto pratico per integrare il linguaggio nei flussi di lavoro tipici dei sistemi operativi (specie in Linux), eseguire script in batch, gestire installazioni di pacchetti e testare rapidamente funzioni o idee.

Culture Pill/esterni

Ecco una piccola "culture pill" su Python:
Il nome "Python" non deriva dal serpente, ma dal gruppo comico inglese "Monty Python"! Guido van Rossum, il creatore del linguaggio, era un fan sfegatato del loro stile surreale e ironico. Per questa ragione, nella community di Python troverai diversi riferimenti divertenti a sketch e battute dei Monty Python: ad esempio, alcuni esempi e tutorial usano termini come "spam", "eggs" e "ham" (riprendendo la famosa scenetta dello "Spam") o fanno apparire battute che ricordano i loro show.

In più, se nel terminale Python scrivi “import this”, compare il cosiddetto “Zen of Python” con 19 principi ironico-filosofici su come scrivere un buon codice: una specie di “manifesto” semiserio del linguaggio. Un altro Easter Egg simpatico è “import antigravity”: apre nel browser una striscia a fumetti sul tema “volare” e su come Python renda più leggere (o meno pesanti) le programmazioni.

Tutte queste piccole curiosità e scherzosità fanno parte della cultura Python!

Spiegazione: “Dynamic typing” and “whitespace” are playful references to Python’s syntax and design. With Python, there’s no need to declare variable types (dynamic typing) and blocks of code are defined by indentation (whitespace) instead of braces or keywords. To someone coming from other languages, these can feel odd or liberating—hence “programming is fun again!”